Progress Report and Results

Sensor Trust for Lunar South Pole Landing - LANTERN testbed, 19 October 2026

Conclusion in plain words

1. A navigation filter with a fixed rule fails on real Site04 terrain exactly like HAKUTO-R: it rejects 99% of valid altimeter readings and the lander crashes (about 500 of 500 runs). **2.** Simply putting the terrain map into the filter fixes most of that. **3.** The biggest further gain comes from not believing the camera when the ground below is in shadow: with it, landing success is 89% instead of about 49%. **4.** The specific four-step altimeter rule (physical-possibility check plus probation) did **not** beat a simpler map-tolerance gate once every method was given the same camera handling (89% versus 93%), and it performs worse when the altimeter is noisy but the scene is well lit (61% versus 99%). **5.** The camera result depends on how badly my simulated camera is hurt by shadow; if shadow hurts it only mildly, the PITFAL-style filter does as well or better. **6.** Delayed altimeter readings defeat every method, and the trust layer's no-map fallback failed. So the honest message is: shadow-aware sensor weighting matters a lot, the four-step altimeter rule adds little beyond a simple tolerance gate, and the evidence is simulation-only.

1. Verified in Unity

Item	Result
Descent	From 102.5 km above a flat spot at Site04 (Unity 6198, 8512; 2.8 deg maximum slope over 45 m): touchdown at t = 583-587 s, 0.50 m/s vertical, 0 m/s horizontal, 0.0 deg tilt, 251 kg fuel left, exactly on target
Live scan (LUMEN)	Fix from 2463 m down. Error 47 m at 2205 m (41 m cell), 14.5 m at 921 m, 4 m at 155 m (8 m cell); median 8.9 m over 108 fixes. No fix below about 80 m or above about 2.5 km
HUD, logging, shadows	HUD text scales with screen height; every run writes CSV to out/runs/; low-Sun cast shadows render
Replay scene	Site04Replay plays one recorded descent with the three filters side by side (true lander solid, filter estimate as a ghost, live altitude error and gate)

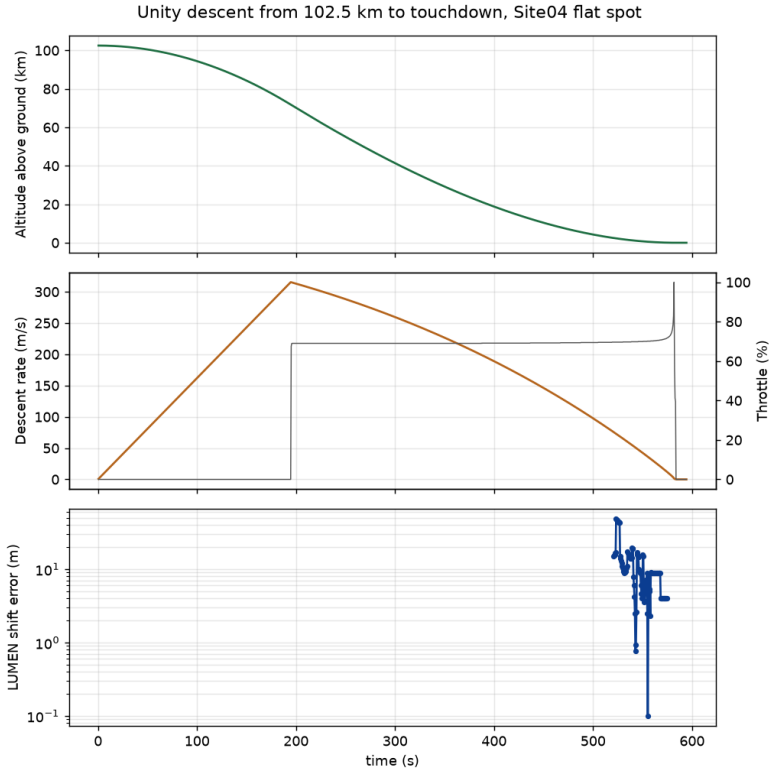


Figure 2. Logged Unity descent from 102.5 km.

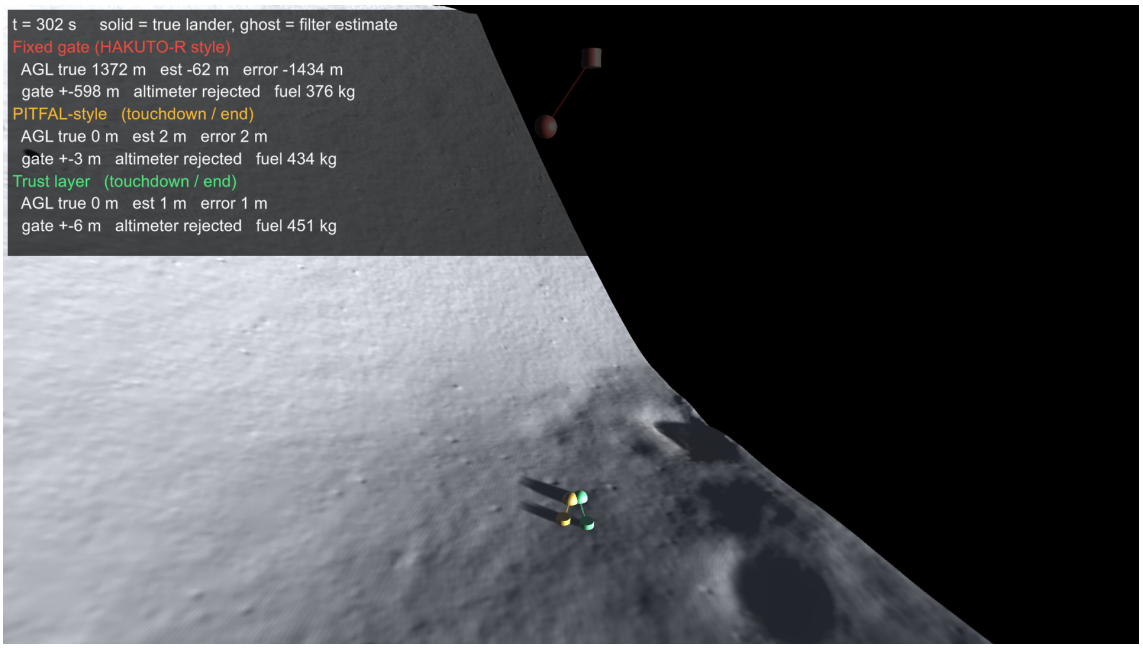


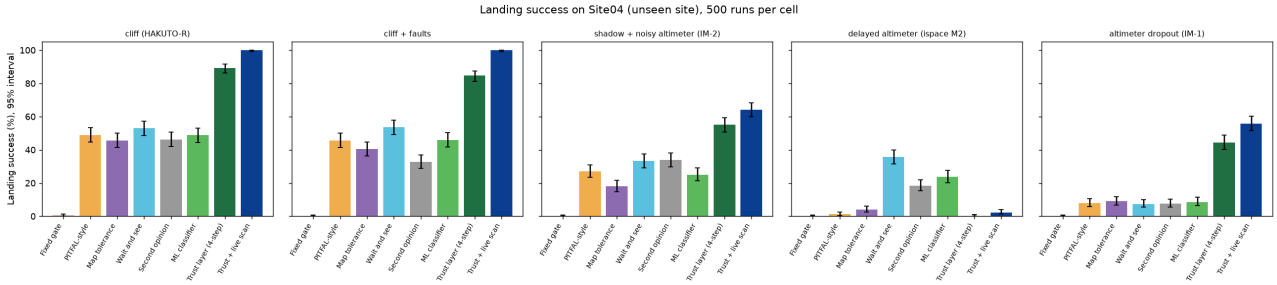
Figure 3. Unity replay of one descent (cliff plus faults). The fixed-gate lander (red) hovers 1372 m above ground believing it is 62 m below the surface; the PITFAL-style (yellow) and trust-layer (green) landers have landed.

2. Monte Carlo results (Python test bench, Site04, never used for tuning)

500 seeded runs per cell. Every method sees the same descents, noise and faults. The ML classifier and the probation settings were trained or tuned on procedural crater worlds only (classifier: 77.7% accuracy, AUC 0.85 on a held-out world; probation window 8 samples, 4 sigma). Sun 2 degrees above the horizon from the west, so the descent enters shadow.

Scenario	Fixed	PITFAL-style	Map tol.	Wait&see;	2nd opinion	ML	Trust 4-step	Trust + scan
Cliff (HAKUTO-R)	0.4	49.0	45.6	53.0	46.2	48.8	89.2	100.0
Cliff + faults	0.0	45.6	40.4	53.6	32.8	46.0	84.6	100.0
Shadow + noisy altimeter (IM-2)	0.0	27.0	18.0	33.2	33.8	25.0	55.0	64.2
Delayed altimeter (ispace M2)	0.0	1.2	4.0	35.6	18.4	23.8	0.2	2.2
Altimeter dropout (IM-1)	0.0	7.8	9.0	7.4	7.6	8.6	44.4	55.8

Table 1. Landing success (%). 95% intervals are about +/-4 points (cliff row: trust 86-92, PITFAL 45-53). Judges other than the trust layer used the camera whether or not the ground was lit.

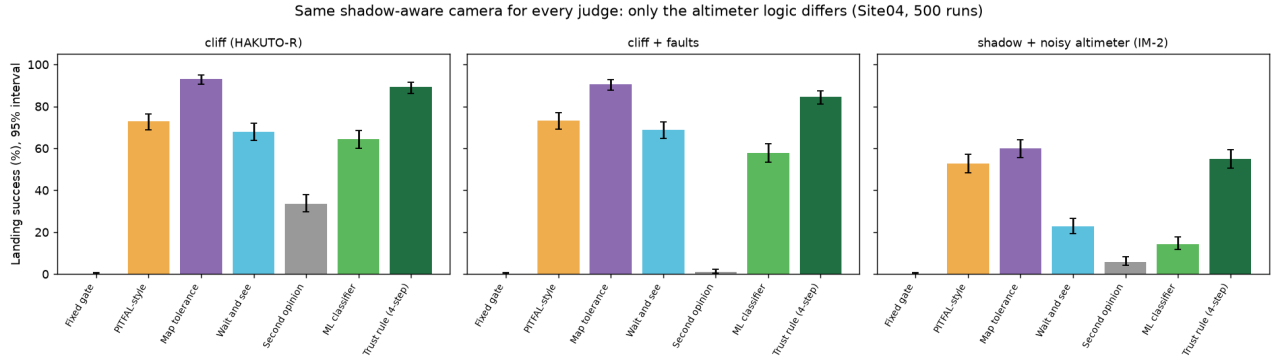


3. Controlled checks that change the interpretation

Table 1 compares the trust layer, which includes shadow-aware camera handling, with judges that lack it. To separate the two ideas, every judge was re-run with the same shadow-aware camera (500 runs):

Scenario	Map tol.	Trust 4-step	PITFAL-style	Wait&see;	ML	2nd opinion
Cliff	93.0	89.2	72.8	68.0	64.4	33.6
Cliff + faults	90.6	84.6	73.2	68.8	57.8	1.0
Shadow + noisy altimeter	60.0	55.0	52.8	22.8	14.4	5.8

Table 2. Landing success (%) with identical camera handling. The simple map-tolerance gate matches or beats the four-step rule.



Ablations (cliff scenario): removing the camera step drops the trust layer from 89% to 47%; removing probation leaves it at 89%; removing the physical-possibility check leaves it at 88%. In the shadow-plus-noise scenario probation helps (55% versus 38% without), while the physical check hurts (63% without it).

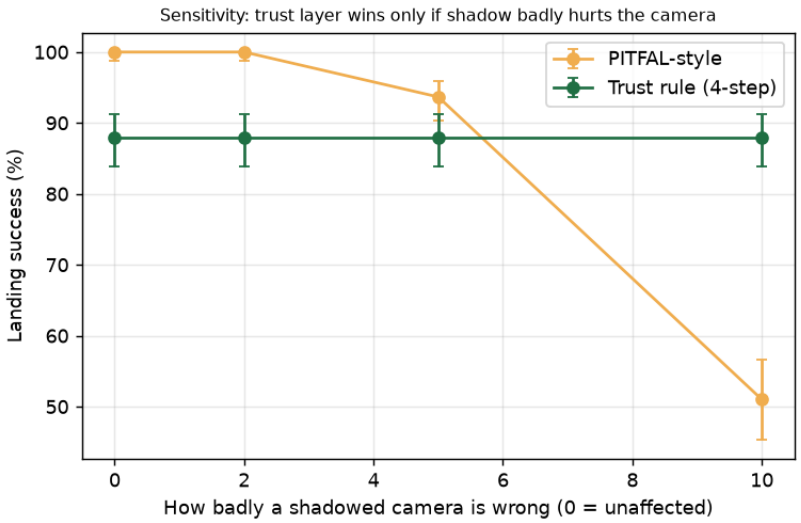
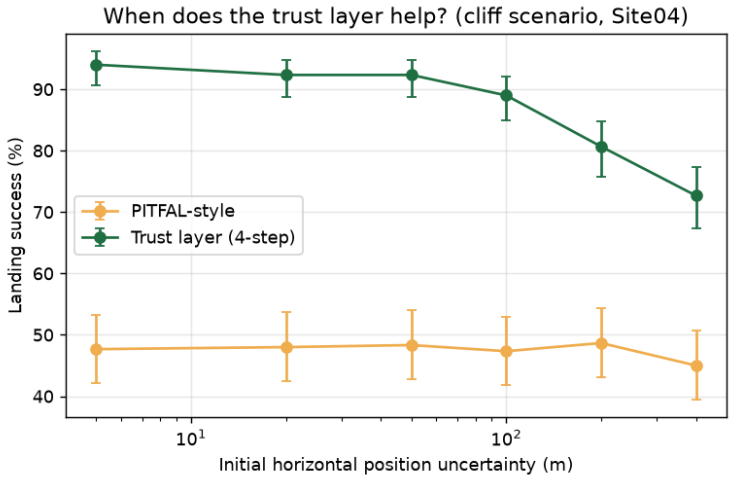


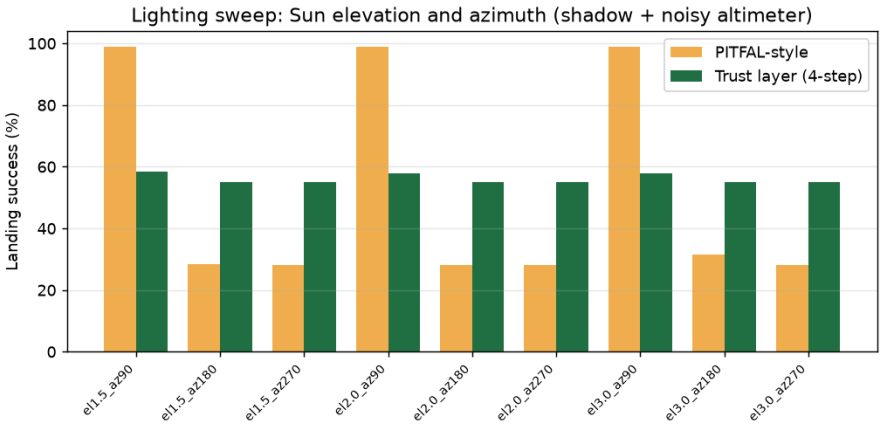
Figure 4. Sensitivity to the camera model. If shadow only mildly degrades the camera, PITFAL-style reaches 94-100% and the trust layer's advantage disappears; the trust layer is flat at 88% because it ignores the camera in the dark.

In a well-lit run (Sun from the east) every map-based judge except the second-opinion one lands at least 98.7% in the cliff scenarios, so the trust layer is not harmful there. With a noisy altimeter in good light, though, PITFAL-style lands 99% and the trust layer only 61%, because probation compares readings to the nominal noise level and rejects noisy but valid ones.

4. Sweeps and robustness (300 runs each unless noted)

Test	PITFAL-style	Trust layer	Reading
Initial position error 5 m / 50 m / 200 m / 400 m (cliff)	48 / 48 / 49 / 45	94 / 92 / 81 / 73	Trust layer gap is largest when position is well known, the opposite of hypothesis H3; the gap is driven by shadow handling
Mid-descent divert	54.3	72.7	Works without re-planning; one test, not tuned
Map shifted by 2 km	17.7	26.0	Both fail; false rejections 70% and 90%
No map	6.7	0.0	The trust layer's cautious fallback does not work (fixed rule: 0)
Unseen site vs tuned site (shadow + noise)	29.3 vs 35.3	51.7 vs 41.7	No drop on the unseen site; the procedural world is simply harder
Sun azimuth 90 degrees (well lit), shadow + noise	99.0	58.0	Trust layer worse in good light with noisy altimeter
Sun azimuth 180 / 270 degrees, elevation 1.5-3 degrees	28 to 31	55.0	Identical for all three elevations: the lander is in shadow throughout





5. Hypotheses against the plan

#	Expectation	Outcome
H1	Four-step rule rejects far fewer real cliff readings than the fixed rule without being fooled noticeably more (at most 1 extra fooled run in 100)	Versus the fixed rule: yes (99% false rejection down to 0.2%). Versus PITFAL-style: false rejection 8.3% down to 0.2%, but missed faults rise from 7.3% to 10.7%, which breaks the stated threshold
H2	Shadow-aware camera trust reduces error in dark areas	Supported in this model (89% versus 47-49%), but depends on how badly shadow hurts the camera; see Figure 4
H3	When position is already well known, the NASA-style method does as well as ours	Not supported here; confounded by the shadow effect

#	Expectation	Outcome
H4	The rule still works on a site it was never tuned on	Supported: 51.7% on Site04 versus 41.7% on the tuned world, with the trust layer ahead of PITFAL on both

6. Limits and honest caveats

- â€¢ Everything is simulation. The shadowed-camera model, the unmapped-terrain roughness and the fault models are my assumptions, and the headline result is sensitive to the camera model (Figure 4).
- â€¢ Guidance, sensor models and the first judge designs were chosen while looking at Site04 smoke runs. Only the probation settings and the ML model were tuned strictly on procedural worlds; the controlled comparisons in section 3 were added after the main run because the first table confounded two ideas.
- â€¢ Site01 is not available, so the tuned site is procedural terrain only. The HAKUTO-R altitude debriefing curve was not available, so no replay of the real data was done.
- â€¢ The Python bench starts at 15 km as the plan specifies; the Unity demonstration starts at 102.5 km from rest at the user's request, which ignores the real orbital speed.
- â€¢ Delayed readings, dead altimeters and a missing map are not solved, as the plan stated for the first two.
- â€¢ The ML classifier is moderate and was no better than PITFAL-style in the tests.

7. Is it publishable?

As a negative-and-mixed result, possibly at a workshop: a controlled study on real Site04 terrain showing that (a) a fixed rule reproduces the HAKUTO-R failure, (b) putting the map in the filter fixes most of it, (c) shadow-aware camera weighting gives the largest further gain but depends on the camera model, and (d) a four-step rule with probation adds little over a map-tolerance gate. To be stronger it would need a better-justified camera model, real image-based navigation, Site01 and a second real site, and a redesign of probation for noisy but valid readings. The data, code and seeds are in the repository.

8. Next steps

- â€¢ **Justify the camera model first** (it decides whether the headline holds): render camera images in Unity under low Sun, measure how a simple landmark matcher degrades with lit fraction, replace the assumed shadow-damage curve, and re-run the sensitivity test.
- â€¢ Redesign probation to learn the noise level from the data, add a time-stamp check for delayed readings, tune on procedural worlds, then run Site04 once with settings frozen beforehand.
- â€¢ Fix the no-map fallback (currently 0%) and re-run the shifted-map and missing-map tests.
- â€¢ Obtain Site01 and PGDA Product 104 for a second real site, and the published HAKUTO-R altitude record for a real-data replay.
- â€¢ Write the workshop-style paper after the first three steps; add per-sensor trust bars and live gate width to the Unity HUD.

9. Repository map

Path	Contents
LANTERN/	Unity 6 project: Site04 and Site04Replay scenes, lander, autopilot, LUMEN, HUD, logger, replay
sim/	Python test bench and pipeline (world.py, engine.py, pipeline.py, plots.py)
out/mc/	Monte Carlo results: main_results.csv/.md, controlled.csv, sweeps, tuning, ML report; per-run CSVs
out/plots/, out/replay/	Result figures; replay data for Unity
docs/	This report and the project description
data/	NASA PGDA GeoTIFFs (not in git; see README)